

Prvi domaći zadatak iz PMT-a

Mina Šljivić

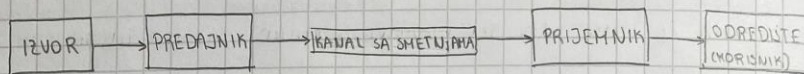
2022/0219

~Prvi domaći zadatak

-TEORIJSKI DEO-

Pitanje 1:

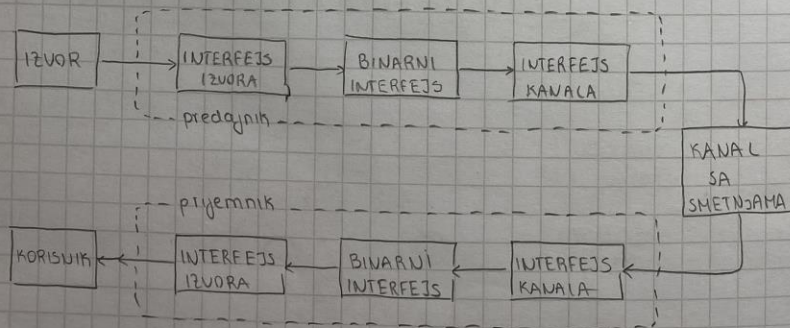
Osnovna blok šema telekom. sistema:



- Predajnik: (transmitter) generiše signale poruke (zvuč, slike, podaci...) i opisuje ih sa određenom preciznošću skupom simbola. Zatim konvertuje signal poruke iz izvora informacija u formu pogodnu za prenos u kanalu.
- Prijemnik: (receiver) vraća signal u originalni oblik i reprodukuje simbole. Rekonstruiše originalni signal poruke sa određenom degradacijom.

Efekte koji se mogu pojaviti u kanalu su slabljenje i izobličenje signala prilikom njihovog prenosa kroz kanal. Šum i smetnje iz drugih izvora se superponiraju na signal na izlazu iz kanala, pa je rezultat signala na izlazu izobličen verzija signala na ulazu.

Opšta blok - šema digitalnog sistema:



Izvor - izvor nekog signala, postoji više podera. Može biti kontinualni / diskretni, analogni / digitalni, deterministički / slučajni.

Interfejs izvora - dizajnira se u skladu sa osobinama izvora. U njemu se vrši digitalizovanje signala pomoću diskretizacije, i pretvaranje u binarni oblik kodovanjem simbola.

Binarni interfejs - odvaja interfejs izvora i kanala. U njemu se vrši obrada i skladištenje podataka koristeći digitalni hardver. Standardizovani binarni interfejs omogućava i spajanje podataka različitih izvora, pošto se svi podaci formiraju kao sekvenca binarnih simbola.

Interfejs kanala - zavisi samo od osobina kanala. On služi da pripremi binarni podatak za prolat kroz kanal. To podrazumeva zaštitu pomoću zaštitnog koda i modulaciju.

Kanal - medijum za prenos informacija (bakarne žice, optički kablovi, atmosfera...)

U njemu se javljaju šum i smetnje iz drugih izvora i uređaja. Javljaju se izobličenja usled ograničenog propusnog opsega ili kao posledica nelinearnosti.

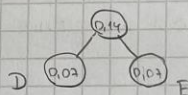
Korisnik - prima poslato signal, koji je po izlasku iz kanala rekonstruisan i vraćen u originalni oblik.

Pitanje 2:

$g=7$ simbola, izvor bez memorije

$$P(D)=0,07, P(E)=0,07, P(A)=0,12, P(C)=0,13, P(F)=0,16, P(G)=0,19, P(B)=0,26$$

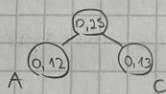
Poredamo simbole po vjerovatnoćama u rastućem poretku, i izdvajamo dva sa najmanjim:



Uzujemo ih sa zajedničkim korenom koji predstavlja njihovu združenu vjerovatnoću.

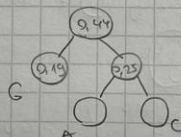
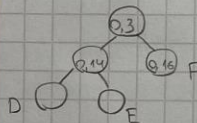
Tu novu, združenu (0,14) vjerovatnoću vidimo u sortirani listi simbola.

$$P(A)=0,12, P(C)=0,13, P(D+E)=0,14, P(F)=0,16, P(G)=0,19, P(B)=0,26$$



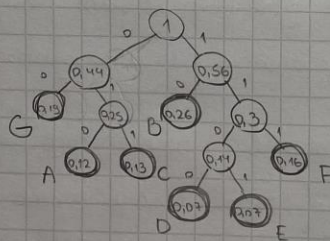
Ponavljamo isti postupak dok ne iskoristimo sve simbole i dobijemo jedno stablo sa korenom sa vjerovatnoćom od 1

$$P(D+E)=0,14, P(F)=0,16, P(G)=0,19, P(A+C)=0,25, P(B)=0,26$$



$$P(B)=0,26, P(D+E+F)=0,3, P(G+A+C)=0,44$$

STABLO:



- Dodamo 0 i 1 kao puteve do čvorova.

- Pročitamo put (niz) 0 i 1 do pojedinačnih čvorova simbola

A: 010
B: 101
C: 011
D: 1100
E: 1101
F: 111
G: 00

Posmatrajuci stablo moze se zakljuciti da li je ono dobijeno

Huffman-ovim postupkom, jer on garantuje trenutni kod, sto vidimo

iz analize da su svi simboli u listovima i nijedan nije na putu do

drugog. Takođe, Huffman-ovo stablo ima "osobinu ladanka"

koja se ogleda u tome da su čvorovi sortirani rastuće učestalosti

posmatramo odozdo-nagore i sleva-nadesno, što je ispunjeno.

$$L_i = \sum_{i=1}^n P(s_i) \cdot l(s_i)$$

$$= 3 \cdot 0,12 + 2 \cdot 0,26 + 3 \cdot 0,13 + 4 \cdot 0,07 + 4 \cdot 0,07 + 3 \cdot 0,16 + 2 \cdot 0,19$$

$$= 2,69 \rightarrow \text{Srednja dužina kodne reči}$$

$$J = \frac{\lceil \lg 9 \rceil}{L_{sr}} = \frac{\lceil \lg 7 \rceil}{2,31} = \frac{3}{2,31} = 1,29 \rightarrow \text{Stepen kompresije}$$

$$H(s) = \sum_{i=1}^n P(s_i) \lg \left(\frac{1}{P(s_i)} \right) = 2,67 \rightarrow J_{\max} = \frac{3}{2,215} = 1,3544$$

Entropija Max. stepen kompresije

$$\eta = \frac{H(s)}{L_{sr}} \cdot 100\% = 99,25\% \rightarrow \text{Efikasnost}$$

Pitanye 3:

Kod sa ponavljanjem n puta se dobija tako što se svaki binarni simbol zameni sa blokom od n tih binarnih simbola:

$n=4$ npr $\rightarrow$ 1 : 1111
0 : 0000

zatim se tako šalje u kanal. U kanalu se dešavaju nasumične greške i pri dekodovanju postoje 2 varijante:

I Ukoliko dobijene sve nule $\rightarrow 0$, sve jedinice $\rightarrow 1$.

Ukoliko se dođe bilo koja druga kombinacija, smatra se da je došlo do greške i ukoliko postoji povratna sprega, zahteva se ponovno slanje reči.

$\sim$ Verovatnoca greške pri dekodovanju $P_e = P^n$ (Greška će biti ne detektovana samo ukoliko se desi na svih n bin simbola)

III Pravilo večinskega odločevanja

Ukoliko u bloku postoji više nula $\rightarrow 0$, više jedinica $\rightarrow 1$

Ova varijanta i detektuje i ispravlja greske

~ Verovatnoća greske:
pri dekodovanju

$$\sum_{t=\frac{n+1}{2}}^n \binom{n}{t} p^t (1-p)^{n-t}$$

$\swarrow$ sansa za toliko pogrešnih bitova
 $\downarrow$ pogrešni bitovi
 $\downarrow$ ispravni bitovi

$$p = 10^{-2}$$

$$n = 5$$

I $p_e = p^5 = (10^{-2})^5 = 10^{-10}$

$$\begin{aligned} \text{II } P_e &= \binom{5}{5} p^5 + \binom{5}{4} p^4 (1-p) + \binom{5}{3} p^3 (1-p)^2 \\ &= p^5 + 5 \cdot p^4 (1-p) + 10 p^3 (1-p)^2 \\ &= 10^{-10} + 4.95 \cdot 10^{-8} + 9.8 \cdot 10^{-6} \end{aligned}$$

Max kodni količnik:

$$R < I_{\max}, \quad I_{\max} = 1 - (1-p) \cdot \lg\left(\frac{1}{1-p}\right) - p \cdot \lg\left(\frac{1}{p}\right)$$

$$p = 10^{-2}$$

$$R < 1 + 0,99 \cdot \lg(0,99) + 0,01 \cdot \lg(0,01)$$

$$R < 1,08$$

↳ Kodni količnik od $R = 1,08$ ^{→ jako malo vr.} će moći da ispravi sve greške

Pitanje 4:

Difi - Helmanov algoritam:

Osoba A i B žele da se dogovore oko ključa, koji će da bude slučajni element u cikličnoj grupi G

→ Oboje imaju informaciju o grupi G i njenom generatoru g

→ Osoba A izabere slučajni broj a iz skupa $G: \{1, 2, \dots, G-1\}$

Posalje osobi B vrednost (g^a)

→ Osoba B izabere slučajni broj b iz skupa $G: \{1, 2, \dots, G-1\}$

Posalje osobi A vrednost (g^b)

→ Osoba A izračuna $(g^b)^a = g^{ab}$
Osoba B izračuna $(g^a)^b = g^{ab}$ } g^{ab} Ključ!

Ovaj algoritam je značajan jer je njime rešen problem nesigurne distribucije ključa, već njegovo generisanje i to u neigurnom kanalu.

Mane: Ukoliko se priključi neka osoba C , i ona šalje informaciju između A i B , može da presretne i ponovo pošalje poruke između A i B , pretvarajući se da je B za A i da je A za B , slanjem svog broja c .
Tako će prevartiti A i B i otkriti tajni ključ.

PRVI ZADATAK (KOD) :

```
#include <stdio.h>
#include <stdint.h>

void print_matrix (int mat[6][10]) {
    printf("Generisuca matrica:\n");
    for (int i = 0; i < 6; i++) {
        printf("| ");
        for (int j = 0; j < 10; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("|\n");
    }
}

int main() {
    int generating_matrix[6][10] = {{1, 1, 1, 0, 0, 0, 0, 0, 0, 0},
                                     {1, 0, 0, 1, 1, 0, 0, 0, 0, 0},
                                     {0, 1, 0, 1, 0, 1, 0, 0, 0, 0},
                                     {1, 1, 0, 1, 0, 0, 1, 0, 0, 0},
                                     {1, 0, 0, 0, 0, 0, 0, 1, 1, 0},
                                     {0, 1, 0, 0, 0, 0, 0, 1, 0, 1}},

    ;

    print_matrix(generating_matrix);
    printf("\nDimenzije matrice su 10x6, pa su to ujedno i
parametri: n = 10, k = 6.\nKodni kolicnik je tada 3/5.\n\n");

    int info_word[6] = {0};
    int code_word[10] = {0};
    int weights[11] = {0};

    int i1 = 0, i2 = 0, i3 = 0, i4 = 0, i5 = 0;
    for (int cnt = 0; cnt < 64; cnt++, i1++, i2++, i3++, i4++,
i5++) {
        if (cnt % 2 == 0) info_word[5] = 0;
        else info_word[5] = 1;

        if (i1 >= 0 && i1 < 2) info_word[4] = 0;
        else if (i1 >= 2 && i1 < 4) info_word[4] = 1;
        if (i1 == 3) i1 = -1;

        if (i2 >= 0 && i2 < 4) info_word[3] = 0;
        else if (i2 >= 4 && i2 < 8) info_word[3] = 1;
    }
}
```

```

    if (i2 == 7) i2 = -1;

    if (i3 >= 0 && i3 < 8) info_word[2] = 0;
    else if (i3 >= 8 && i3 < 16) info_word[2] = 1;
    if (i3 == 15) i3 = -1;

    if (i4 >= 0 && i4 < 16) info_word[1] = 0;
    else if (i4 >= 16 && i4 < 32) info_word[1] = 1;
    if (i4 == 31) i4 = -1;

    if (i5 >= 0 && i5 < 32) info_word[0] = 0;
    else if (i5 >= 32 && i5 < 64) info_word[0] = 1;

    int weight = 0;
    for (int j = 0; j < 10; j++) {
        code_word[j] = 0;
        for (int k = 0; k < 6; k++) {

            code_word[j] = (code_word[j] + info_word[k] *
generating_matrix[k][j]) % 2;
        }
        if (code_word[j] == 1) weight ++;
        printf("%d", code_word[j]);
    }
    weights[weight] ++;
    printf("    : (%d)\n", weight);
}

int d = 10;
for (int k = 0; k <= 10; k++) {
    if (weights[k] < d && weights[k] > 0 && k != 0) d =
weights[k];
    printf("\nTezina %d: %d reci.", k, weights[k]);
}

printf ("\n\nMinimalno Hemingovo rastojanje je %d.\n", d);
int ec = (d - 1) / 2;
printf ("Broj gresaka koje kod moze da ispravi je %d.\n", ec);

return 0;
}

```

PRVI ZADATAK (IZLAZ):

```
Run: PMT_dz1 x
C:\code\c\PMT_domaci1\zadatak1\cmake-build-debug\PMT_dz1.exe
Generisuci matrica:
| 1 1 1 0 0 0 0 0 0 0 |
| 1 0 0 1 1 0 0 0 0 0 |
| 0 1 0 1 0 1 0 0 0 0 |
| 1 1 0 1 0 0 1 0 0 0 |
| 1 0 0 0 0 0 0 1 1 0 |
| 0 1 0 0 0 0 0 1 0 1 |

Dimenzije matrice su 10x6, pa su to ujedno i parametri: n = 10, k = 6.
Kodni kolicnik je tada 3/5.

0000000000 : (0)
0100000101 : (3)
1000000110 : (3)
1100000011 : (4)
1101001000 : (4)
1001001101 : (5)
0101001110 : (5)
0001001011 : (4)
0101010000 : (3)
0001010101 : (4)
1101010110 : (6)
1001010011 : (5)
1000011000 : (3)
1100011101 : (6)
0000011110 : (4)
0100011011 : (5)
1001100000 : (3)
1101100101 : (6)
0001100110 : (4)
0101100011 : (5)
0100101000 : (3)
0000101101 : (4)
1100101110 : (6)

0000101101 : (4)
1100101110 : (6)
1000101011 : (5)
1100110000 : (4)
1000110101 : (5)
0100110110 : (5)
0000110011 : (4)
0001111000 : (4)
0101111101 : (7)
1001111110 : (7)
1101111011 : (8)
1110000000 : (3)
1010000101 : (4)
0110000110 : (4)
0010000011 : (3)
0011001000 : (3)
0111001101 : (6)
1011001110 : (6)
1111001011 : (7)
1011010000 : (4)
1111010101 : (7)
0011010110 : (5)
0111010011 : (6)
0110011000 : (4)
0010011101 : (5)
1110011110 : (7)
1010011011 : (6)
0111100000 : (4)
0011100101 : (5)
1111100110 : (7)
1011100011 : (6)
1010101000 : (4)
1110101101 : (7)
0010101110 : (5)
0110101011 : (6)
```

➤ 0010110000 : (3)
0110110101 : (6)
1010110110 : (6)
1110110011 : (7)
1111111000 : (7)
1011111101 : (8)
0111111110 : (8)
0011111011 : (7)

Tezina 0: 1 reci.
Tezina 1: 0 reci.
Tezina 2: 0 reci.
Tezina 3: 10 reci.
Tezina 4: 16 reci.
Tezina 5: 12 reci.
Tezina 6: 12 reci.
Tezina 7: 10 reci.
Tezina 8: 3 reci.
Tezina 9: 0 reci.
Tezina 10: 0 reci.

Minimalno Hemingovo rastojanje je 3.
Broj gresaka koje kod moze da ispravi je 1.

Process finished with exit code 0

DRUGI ZADATAK (KOD):

```
#include <stdio.h>
#include <stdlib.h>
#define LEN 10000

void print_matrix (int mat[6][10]) {
    printf("Generisuca matrica:\n");
    for (int i = 0; i < 6; i++) {
        printf("| ");
        for (int j = 0; j < 10; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("|\n");
    }
}

int main() {
    int generating_matrix[6][10] = {{1, 1, 1, 0, 0, 0, 0, 0, 0, 0},
                                     {1, 0, 0, 1, 1, 0, 0, 0, 0, 0},
                                     {0, 1, 0, 1, 0, 1, 0, 0, 0, 0},
                                     {1, 1, 0, 1, 0, 0, 1, 0, 0, 0},
                                     {1, 0, 0, 0, 0, 0, 0, 1, 1, 0},
                                     {0, 1, 0, 0, 0, 0, 0, 1, 0, 1}},

    print_matrix(generating_matrix);

    printf("\nPrvih 10 random generisanih informacionih reci:\n");
    int info_words[1000][6] = { 0 };
    int code_words[1000][10] = { 0 };

    for (int i = 0; i < 1000; i++) {
        if (i < 10) printf("Inf rec%2d: ", i+1);
        for (int j = 0; j < 6; j++) {
            info_words[i][j] = rand() % 2;
            if (i < 10) printf("%d", info_words[i][j]);
        }
        if (i < 10) printf("\n");
    }

    printf("\nNjihove odgovarajuce kodne reci:\n");
    for (int i = 0; i < 1000; i++) {
        if (i < 10) printf("Kod rec%2d: ", i+1);
        for (int j = 0; j < 10; j++) {
            for (int k = 0; k < 6; k++) {

                code_words[i][j] = (code_words[i][j] +
```

```

info_words[i][k] * generating_matrix[k][j]) % 2;
    }
    if (i < 10) printf("%d", code_words[i][j]);
}
if (i < 10) printf("\n");
}

int interleaver[LEN], cnt = 0;
for (int j = 0; j < 10; j++)
{
    for (int i = 0; i < 1000; i++)
    {
        interleaver[cnt] = code_words[i][j];
        cnt++;
    }
}
int error;
printf ("\nUnesite poziciju prve od paketa od 3 greske: ");
scanf ("%d", &error);

for (int i = error; i < LEN; i += 50)
{
    interleaver[i] = (interleaver[i] + 1) % 2;
    interleaver[i + 1] = (interleaver[i + 1] + 1) % 2;
    interleaver[i + 2] = (interleaver[i + 2] + 1) % 2;
}

printf("\nKodne reci nakon interlivinga:\n");
cnt = 0;
for (int j = 0; j < 10; j++) {
    for (int i = 0; i < 1000; i++) {
        code_words[i][j] = interleaver[cnt];
        cnt++;
    }
}
for (int i = 0; i < 1000; i++) {
    if (i < 10) printf("Kod rec%2d: ", i+1);
    for (int j = 0; j < 10; j++) {
        if (i < 10) printf("%d", code_words[i][j]);
    }
    if (i < 10) printf("\n");
}

printf("\nDekodirane informacione reci nakon interlivinga:\n");
int new_info[1000][6] = { 0 };

for (int i = 0; i < 1000; i++) {
    if (i < 10) printf("Info rec%2d: ", i+1);

```

```

        for (int j = 0, k = 0; j < 6 && k < 10; k++) {
            if (k != 0 && k != 1 && k != 3 && k != 7) {
                new_info[i][j] = code_words[i][k];
                if (i < 10) printf("%d", new_info[i][j]);
                j++;
            }
        }
        if (i < 10) printf("\n");
    }

    float num_of_diff = 0;
    for (int i = 0; i < 1000; i++) {
        for (int j = 0; j < 6; j++) {
            if (new_info[i][j] != info_words[i][j]) {
                num_of_diff++;
                break;
            }
        }
    }
    printf ("\nVerovatnoca greske po bitu: %.2f", num_of_diff/10);
    putchar('%');
    return 0;
}

```

DRUGI ZADATAK (IZLAZ):

Run:  zadatak2 x
C:\code\c\PMT_domaci1\zadatak2\cmake-build-debug\zadatak2.exe

 Generisuca matrica:

 | 1 1 1 0 0 0 0 0 0 0 |
| 1 0 0 1 1 0 0 0 0 0 |
| 0 1 0 1 0 1 0 0 0 0 |
 | 1 1 0 1 0 0 1 0 0 0 |
 | 1 0 0 0 0 0 0 1 1 0 |
 | 0 1 0 0 0 0 0 1 0 1 |

 Prvih 10 random generisanih informacionih reci:

Inf rec 1: 110010
Inf rec 2: 000011
Inf rec 3: 111110
Inf rec 4: 101001
Inf rec 5: 001001
Inf rec 6: 101010
Inf rec 7: 111011
Inf rec 8: 011011
Inf rec 9: 101001
Inf rec10: 111110

Njihove odgovarajuće kodne reci:

Kod rec 1: 1111100110
Kod rec 2: 1100000011

Kod rec 3: 0111111110
Kod rec 4: 1111010101
Kod rec 5: 0001010101
Kod rec 6: 0011010110
Kod rec 7: 1110110011
Kod rec 8: 0000110011
Kod rec 9: 1111010101
Kod rec10: 0111111110

 Unesite poziciju prve od paketa od 3 greske: 6

```

■ Kodne reci nakon interlivinga:
Kod rec 1: 1111100110
Kod rec 2: 1100000011
Kod rec 3: 0111111110
Kod rec 4: 1111010101
Kod rec 5: 0001010101
Kod rec 6: 0011010110
Kod rec 7: 0001001100
Kod rec 8: 1111001100
Kod rec 9: 0000101010
Kod rec10: 0111111110

Dekodirane informacione reci nakon interlivinga:
Info rec 1: 110010
Info rec 2: 000011
Info rec 3: 111110
Info rec 4: 101001
Info rec 5: 001001
Info rec 6: 101010
Info rec 7: 000100
Info rec 8: 100100
Info rec 9: 010110
Info rec10: 111110

Verovatnoca greske po bitu: 6.00%
Process finished with exit code 0

```

Dokle god je velicina paketske greske manja od parametara interlivera (broj vrsta tj. duzina kodne reci, i kolona) greske ce moci da se isprave. Tada ce se originalna rec sa greskom izdeliti tako da u svakoj reci sada zavrsi samo po jedna greska, a kako je dokazano u prathodnom zadtku, kod sa parametrima (10, 6) moze ispraviti jednu gresku. Broj kolona moze varirati i zavisi od broja reci.



U intreliveru:

Sada u svakoj „novoj reci“ postoji po samo jedna greska, koja moze da se ispravi.